

Strukturalni paterni

U ovom dokumentu navedeni su strukturalni paterni korišteni u modifikaciji trenutnog dijagrama sistema klasa i razlozi za njihov odabir.

1. Facade patern - KupoprodajaFacade

Problem

U sistemu za kupoprodaju i iznajmljivanje artikala, klijentski kod (npr. kontroler u prezentacijskom sloju) koji želi izvršiti operaciju kupovine mora samostalno koordinirati između više klasa: provjeriti dostupnost `Artikal` objekta, kreirati `Transakcija` objekat, eventualno kreirati `Rezervacija` objekat, te validirati `Korisnik` koji vrši transakciju. Ovo znači da klijent mora poznavati internu strukturu svih tih klasa i njihove međusobne zavisnosti, što rezultira čvrstom spregom (tight coupling) između prezentacijskog sloja i poslovne logike.

Rješenje

Uvedena je klasa `KupoprodajaFacade` koja implementira interfejs `ITransakcijaFacade` i nudi samo dvije jasne, visokonivojske metode prema klijentu: `kupiArtikl()` i `iznajmiArtikl()`. Unutar sebe, Facade interno koordinira sve potrebne korake - provjerava korisnika, provjerava dostupnost artikla, kreira transakciju i po potrebi kreira rezervaciju. Klijent ne zna ništa o tim klasama i ne ovisi direktno o njima, već komunicira isključivo kroz Facade interfejs.

Zašto je primjenljiv u ovom sistemu

Sistem već razlikuje dva tipa transakcije kroz `TipTransakcijeEnum` (prodaja i iznajmljivanje), što znači da logika unutar `kupujArtikl()` i `iznajmiArtikl()` prirodno postoji u domeni - Facade je samo organizuje na jednom mjestu. Bez Facade paterna, svaki novi klijent (mobilna aplikacija, web interfejs) morao bi sam reimplementirati tu koordinacijsku logiku.

Konsekvence

Facade smanjuje spregu između slojeva sistema i čini prezentacijski sloj nezavisnim od promjena u poslovnoj logici. Ako se npr. proces kreiranja rezervacije promijeni, mijenja se samo Facade, a ne svaki klijent koji ga koristi. Mana je što Facade može postati tzv. "God object" ako se previše logike akumulira u njemu, pa je važno da samo delegira pozive a ne sadrži kompleksnu logiku sam.

2. Proxy patern - ArtiklServisProxy

Problem

Klasa `Artikal` i operacije nad njom (dodavanje, uređivanje, uklanjanje) nisu dozvoljene svim korisnicima jednako - sistem ima `UlogaEnum` koji razlikuje kupce, prodavače i moderatore, te `aktivan` atribut na `Korisnik` klasi. Međutim, direktan pristup servisnoj logici ne nudi nikakav mehanizam kontrole pristupa. Svaki pozivalac može pozvati bilo koju metodu bez provjere.

Rješenje

Uveden je interfejs `IArtikalServis` koji definira ugovor za sve operacije nad artiklima. Ovaj interfejs implementiraju dvije klase. `ArtikalServisImpl` je tzv. RealSubject - stvarna implementacija koja izvršava operacije nad `Artikal` objektima. `ArtikalServisProxy` je Proxy koji stoji ispred `ArtikalServisImpl`-a: svaki poziv metode prvo prolazi kroz privatnu metodu `provjeriPristup(korisnik)` koja čita `uloga` i `aktivan` attribute iz `Korisnik` objekta. Tek ako je pristup dozvoljen, Proxy delegira poziv pravoj implementaciji. Klijent uvijek razgovara samo sa `IArtikalServis` interfejsom i ne zna da li koristi Proxy ili direktnu implementaciju.

Zašto je primjenljiv u ovom sistemu

Sistem već posjeduje sve potrebne elemente za ovaj patern: `UlogaEnum` definira uloge korisnika, `aktivan: boolean` na `Korisnik` klasi omogućava provjeru aktivnosti, a `ZahtjevZaUklanjanje` i `AdminModerator` klase jasno ukazuju da različiti korisnici imaju različita prava nad artiklima. Proxy centralizuje te provjere na jednom mjestu umjesto da se ponavljaju kroz cijelu aplikaciju.

Konsekvence

Proxy patern omogućava dodavanje kontrole pristupa bez ikakve izmjene u `ArtikalServisImpl` klasi, što je u skladu sa Open/Closed principom - klase su otvorene za proširenje, ali zatvorene za modifikaciju. Klijentski kod se također ne mijenja jer i dalje koristi isti `IArtikalServis` interfejs. Mana je uvođenje dodatnog sloja indirekcije koji blago povećava kompleksnost sistema, ali ta cijena je opravdana sigurnošću i konzistentnošću kontrole pristupa.

3. Adapter patern

Problem

Sistem u budućnosti može imati potrebu za integracijom sa eksternim servisima - npr. eksternim servisom za plaćanje (PayPal, Stripe) ili eksternim servisom za slanje notifikacija (email, SMS). Ti eksterni servisi imaju vlastite interfejske koji nisu kompatibilni sa interfejsima koje naš sistem očekuje. Direktna integracija bi značila mijenjanje postojećih klasa poput `Transakcija` ili `Korisnik` svaki put kada se promijeni eksterni servis, što narušava stabilnost sistema.

Rješenje

Uveo bi se interfejs npr. `IPlacanje` koji naš sistem poznaje i koristi. Za svaki eksterni servis kreirala bi se Adapter klasa, npr. `PayPalAdapter` ili `StripeAdapter`, koja implementira `IPlacanje` interfejs a interno poziva metode eksternog servisa prevodeći pozive iz formata našeg sistema u format koji eksterni servis razumije. Klasa `Transakcija` bi koristila isključivo `IPlacanje` interfejs i ne bi znala ništa o konkretnom eksternom servisu iza njega.

Zašto je primjenjiv u ovom sistemu

Klasa `Transakcija` već sadrži attribute poput `iznos` i `valuta` koji su direktno vezani za proces plaćanja. Kako sistem raste i uvodi više načina plaćanja, Adapter patern omogućava dodavanje novih načina plaćanja bez ikakvog diranja u postojeće klase. Svaki novi servis za plaćanje dobija samo svoju Adapter klasu.

Konsekvence

Adapter patern omogućava da sistem bude neovisan od konkretnih eksternih implementacija, što olakšava zamjenu jednog servisa drugim. Mana je što svaki novi eksterni servis zahtijeva kreiranje nove Adapter klase, što može povećati broj klasa u sistemu, ali je ta cijena opravdana fleksibilnošću i izolacijom promjena.

4. Bridge patern

Problem

U sistemu postoje različite vrste artikala (npr. vozila, nekretnine, elektronika) i različiti načini njihovog prikazivanja ili obrade (prikaz u listi, prikaz u detalju, prikaz na mapi). Ako bi se svaka kombinacija vrste artikla i načina prikaza implementirala kao zasebna klasa, broj klasa bi eksponencijalno rastao - `VoziloListaPrikaz`, `VoziloDetaljPrikaz`, `NekretninaListaPrikaz` itd. Svaka nova vrsta artikla ili novi način prikaza zahtijevao bi kreiranje čitavog novog seta klasa.

Rješenje

Bridge patern razdvaja dvije hijerarhije u zasebne klase koje se razvijaju nezavisno. Apstrakcija bi bila `Artikal` sa podklasama po vrsti (npr. `VoziloArtikl`, `NekretninaArtikl`), dok bi implementacija bila posebna hijerarhija `IPrikazArtikla` sa konkretnim klasama poput `ListaPrikaz` i `DetaljPrikaz`. `Artikal` bi držao referencu na `IPrikazArtikla` interfejs i delegirao mu odgovornost prikaza, umjesto da sam zna kako se prikazuje.

Zašto je primjenjiv u ovom sistemu

Klasa `Artikal` već ima attribute poput `kategorija` i `tipArtikla` koji nagovještavaju da će u sistemu postojati različite vrste artikala. Istovremeno, sistem ima i `SlikaArtikla` klasu koja je već odvojena od same logike artikla, što pokazuje tendenciju sistema ka razdvajanju prikaza od podataka. Bridge patern bi formalizovao taj odnos i proširio ga na cjelokupnu logiku prikaza.

Konsekvence

Bridge patern smanjuje broj potrebnih klasa i omogućava da se nove vrste artikala i novi načini prikaza dodaju potpuno nezavisno jedni od drugih. Mana je povećana inicijalna kompleksnost dizajna jer zahtijeva dobro promišljenu podjelu odgovornosti između dvije hijerarhije, što može biti pretjerano za manji sistem.

5. Composite patern

Problem

U sistemu artikli mogu biti organizirani u kategorije, a kategorije mogu imati podkategorije - npr. "Vozila" → "Automobili" → "Polovni automobili". Klijentski kod koji treba prikazati sve artikle unutar kategorije mora posebno tretirati kategorije koje imaju podkategorije i kategorije koje direktno sadrže artikle, što rezultira složenim `if/else` ili `switch` blokovima i otežava dodavanje novih nivoa hijerarhije.

Rješenje

Uveo bi se zajednički interfejs npr. `IKatalogStavka` koji implementiraju i `Artikal` (list node - nema djece) i `Kategorija` (composite node — može sadržavati druge `IKatalogStavka` objekte, bilo artikle ili podkategorije). Klasa `Kategorija` bi interno čuvala listu `IKatalogStavka` objekata i pri pozivu metode poput `prikazi()` ili `pretrazi()` rekurzivno pozivala istu metodu na svakom svom djetetu. Klijent uvijek radi samo sa `IKatalogStavka` interfejsom i ne razlikuje da li radi sa jednim artiklom ili čitavom kategorijom.

Zašto je primjenjiv u ovom sistemu

Klasa `Artikal` već ima atribut `kategorija` što direktno ukazuje na hijerarhijsku organizaciju artikala u sistemu. Kako sistem raste i broj kategorija se povećava, Composite patern omogućava uniformno upravljanje čitavom hijerarhijom bez posebnog tretmana pojedinih nivoa. Operacije poput pretrage ili filtriranja po kategoriji postaju trivijalne jer se automatski propagiraju kroz hijerarhiju.

Konsekvence

Composite patern čini klijentski kod znatno jednostavnijim jer ne mora razlikovati lisne od kompozitnih čvorova. Dodavanje novih nivoa hijerarhije kategorija ne zahtijeva nikakve izmjene u klijentskom kodu. Mana je što interfejs može postati previše generičan - operacije koje imaju smisla za `Kategorija` ne moraju imati smisla za `Artikal`, pa implementacija interfejsa može biti neprirodan fit za lisne čvorove.

6. Decorator patern

Problem

Artikli u sistemu mogu imati različite dodatne opcije koje prodavač može uključiti — npr. "istaknuti oglas" koji se prikazuje na vrhu liste, "hitna prodaja" koja dodaje posebnu oznaku, ili "verifikovani artikl" koji prikazuje badge povjerenja. Implementacija svih ovih kombinacija kao zasebnih podklasa (`IstaknutiArtikl`, `HitnaProduktArtikl`, `IstaknutiHitniArtikl` itd.) bila bi neodrživa jer broj kombinacija eksponencijalno raste sa svakom novom opcijom.

Rješenje

Uveo bi se `ArtikalDecorator` kao apstraktna klasa koja implementira isti interfejs kao i `Artikal` i interno drži referencu na `Artikal` objekat koji dekorira. Konkretni dekoratori poput `IstaknutiArtikalDecorator` ili `VerifikovaniArtikalDecorator` nasljeđuju `ArtikalDecorator`, dodaju svoju specifičnu funkcionalnost (npr. dodavanje oznake ili promjena pozicije u listi) i delegiraju ostale pozive originalnom `Artikal` objektu. Dekoratori se mogu slagati jedan na drugi u bilo kojoj kombinaciji.

Zašto je primjenjiv u ovom sistemu

Sistem već ima `Ocjena` i `Primjedba` klase koje dinamički obogaćuju artikal dodatnim informacijama, a `AdminModerator` i `ZahtjevZaUklanjanje` ukazuju da artikli prolaze kroz različite statusne promjene tokom svog životnog vijeka. Decorator patern bi formalizovao mehanizam dinamičkog dodavanja novih ponašanja artiklima bez nasljeđivanja, što je posebno korisno jer se kombinacije opcija mogu mijenjati u toku izvršavanja programa prema izboru prodavača.

Konsekvence

Decorator patern eliminiše eksploziju podklasa i omogućava dinamičko dodavanje i uklanjanje funkcionalnosti u toku izvršavanja, za razliku od statičnog nasljeđivanja. Mana je što sistem dekoratora može biti teže za debugiranje jer se pozivi propagiraju kroz više slojeva dekoratora, a redosljed kojim se dekoratori slažu može utjecati na krajnji rezultat.